Anisotropic Translation/Rotation Speed Field for Shape-NTFields: Implementation Notes

Summary

These notes describe how to replace the isotropic Eikonal constraint in Shape-NTFields with a block-anisotropic one, in which translation and rotation have separate, pose-dependent speeds $S_t(g)$ and $S_r(g)$. Relative to the existing code, three things change:

1. the gradient of T is taken as a *body-frame Lie gradient* on $SE(3)$ (Section 2);
2. the Eikonal residual becomes a block-weighted norm of that gradient (Section 3);
3. path extraction uses the corresponding block-weighted descent velocity (Section 5).

The network architecture, the computation of S_t and S_r, sampling, and all other loss terms are unchanged. Every formula below was checked numerically against finite differences (Section 6).

1 Model

Conventions. A pose is $g = (R, p) \in SE(3)$. A body twist is $\xi = (v, \omega) \in \mathbb{R}^6$, with **translation first**: v is the body-frame linear velocity and ω the body-frame angular velocity. The pose evolves by $\dot{g} = g\,\hat{\xi}$, i.e. $\dot{p} = R\,v$ and $\dot{R} = R\,[\omega]_\times$.

Time cost. With a translational speed $S_t(g)$ and a rotational speed $S_r(g)$, the time to make a small motion $d\xi = (dv, d\omega)$ is

$$dt = \sqrt{\frac{\|dv\|^2}{S_t(g)^2} + \frac{\|d\omega\|^2}{S_r(g)^2}}, \tag{1}$$

which corresponds to the pose-dependent block weight

$$W(g) = \begin{pmatrix} \dfrac{1}{S_t(g)^2} I_3 & 0 \\ 0 & \dfrac{1}{S_r(g)^2} I_3 \end{pmatrix}, \qquad W(g)^{-1} = \begin{pmatrix} S_t(g)^2 I_3 & 0 \\ 0 & S_r(g)^2 I_3 \end{pmatrix}. \tag{2}$$

The set of velocities reachable in unit time is an ellipsoid with semi-axes S_t (three translational axes) and S_r (three rotational axes).

Eikonal equation. Let $T(g)$ be the minimum travel time from a fixed pose. Then

$$\boxed{\sqrt{S_t(g)^2\,\|\nabla_v T\|^2 + S_r(g)^2\,\|\nabla_\omega T\|^2} = 1} \tag{3}$$

where $\nabla_v T, \nabla_\omega T \in \mathbb{R}^3$ are the translational and rotational halves of the body-frame Lie gradient defined next. When $S_t = S_r = S$ this reduces to the isotropic $S\,\|\nabla T\| = 1$. A derivation sketch is in Appendix A.

2 The Lie gradient

2.1 Definition

For the basis twists $e_1, \dots, e_6$ (translation first),

$$E_i T(g) = \frac{d}{d\varepsilon}\Big|_{\varepsilon=0} T\big(g \exp(\varepsilon\, \hat{e}_i)\big), \qquad \nabla_v T = (E_1 T, E_2 T, E_3 T), \quad \nabla_\omega T = (E_4 T, E_5 T, E_6 T). \tag{4}$$

These are the components of the directional derivative $\xi \mapsto \frac{d}{d\varepsilon} T(g \exp(\varepsilon \hat{\xi}))$ in the body-frame basis; they replace per-coordinate partial derivatives, which are not well defined on $SE(3)$ independently of the rotation parametrization.

2.2 Structure that simplifies the implementation

Right-multiplying by the generators gives

$$g \exp(\varepsilon\, \hat{e}_j) = (R,\ p + \varepsilon\, R\, e_j) \quad (j = 1, 2, 3), \qquad g \exp(\varepsilon\, \hat{e}_{3+j}) = \big(R \exp(\varepsilon [e_j]_\times),\ p\big). \tag{5}$$

Rotational generators leave p fixed, and translational generators move p along the body axes. Two consequences follow.

Translation block: no change from the existing code.

$$\nabla_v T = R^\top \frac{\partial T}{\partial p}, \qquad \text{so} \qquad \|\nabla_v T\| = \Big\| \frac{\partial T}{\partial p} \Big\|. \tag{6}$$

Since only the norm enters (3), the ordinary autograd gradient with respect to the world-frame position suffices.

Rotation block: depends on how the network receives R. Use whichever matches the input encoding.

- **Unit quaternion input** $q = (w, x, y, z)$, Hamilton convention. With $\partial T / \partial q \in \mathbb{R}^4$ from autograd,

$$\nabla_\omega T = G(q)^\top \frac{\partial T}{\partial q}, \qquad G(q) = \frac{1}{2} \begin{pmatrix} -x & -y & -z \\ w & -z & y \\ z & w & -x \\ -y & x & w \end{pmatrix}. \tag{7}$$

  The columns of $G(q)$ are orthogonal to q, so any radial (norm-changing) component of $\partial T / \partial q$ is discarded automatically. The formula gives the same result at q and $-q$.

- **Rotation-matrix input** (e.g. flattened 3×3 or its first two columns). With $\partial T / \partial R \in \mathbb{R}^{3\times 3}$ from autograd,

$$\nabla_\omega T = \mathrm{vee}\big(M - M^\top\big), \qquad M = R^\top \frac{\partial T}{\partial R}, \qquad \mathrm{vee}(S) = (S_{32},\ S_{13},\ S_{21}). \tag{8}$$

- **Any other encoding** (Euler angles, 6D, ...). Insert a zero perturbation $\delta \in \mathbb{R}^3$ upstream of the encoding and differentiate at $\delta = 0$:

$$\nabla_\omega T = \frac{\partial}{\partial \delta}\Big|_{\delta=0} T\big(R\,(I + [\delta]_\times),\ p\big). \tag{9}$$

  The first-order factor $I + [\delta]_\times$ has exactly the same derivative at $\delta = 0$ as $\exp([\delta]_\times)$, and it avoids the NaN gradient that Rodrigues' formula produces at zero angle.

2.3 Both endpoints

$T(g_s, g_g)$ depends on a start and a goal pose. Because the time cost is symmetric (the reverse path costs the same), (3) holds in *each* argument, with the speeds evaluated at that argument:

$$\sqrt{S_t(g_s)^2\|\nabla_v^s T\|^2 + S_r(g_s)^2\|\nabla_\omega^s T\|^2} = 1, \qquad \sqrt{S_t(g_g)^2\|\nabla_v^g T\|^2 + S_r(g_g)^2\|\nabla_\omega^g T\|^2} = 1. \tag{10}$$

Compute the Lie gradient for the start slice and the goal slice of the input separately, exactly as the existing code does for the isotropic gradient.

3 Loss

Define the residual at a pose

$$N(g) = \sqrt{S_t(g)^2\,\|\nabla_v T\|^2 + S_r(g)^2\,\|\nabla_\omega T\|^2}, \qquad \text{target } N(g) = 1. \tag{11}$$

Wherever the current code forms the isotropic quantity $S\,\|\nabla T\|$, substitute N, and keep the existing residual-to-loss mapping.

If the code compares a predicted speed to a ground-truth speed. Split N into an overall speed and a trade-off weight:

$$s_{\text{true}} = \sqrt{S_t^2 + S_r^2}, \qquad \alpha = \frac{S_t^2}{s_{\text{true}}^2}, \qquad s_{\text{pred}} = \frac{1}{\sqrt{\alpha\,\|\nabla_v T\|^2 + (1-\alpha)\,\|\nabla_\omega T\|^2}}. \tag{12}$$

Then $s_{\text{true}}/s_{\text{pred}} = N$, so any existing loss of the form $\ell(s_{\text{pred}}, s_{\text{true}})$ can be reused unchanged. Here s_{true} carries "how close to an obstacle" and $\alpha \in (0,1)$ carries "translate versus rotate": $\alpha \to 0$ means rotation is easy and translation is blocked.

Second-order gradients. The loss depends on ∇T, so all gradient calls inside the loss need `create_graph=True`, as in the isotropic version.

4 Reference code (PyTorch)

The input layout is codebase-specific. The snippet assumes each pose is a slice `[p(3), q(4)]` of the network input `x`; adjust the marked slices to the actual layout.

```
import torch

def rot_grad_quat(q, dT_dq):
    """Body-frame rotational Lie gradient from a quaternion gradient.
    q: (B,4) as (w,x,y,z), Hamilton.  dT_dq: (B,4).  Returns (B,3)."""
    w, x, y, z = q.unbind(-1)
    G = 0.5 * torch.stack([
        torch.stack([-x, -y, -z], -1),
        torch.stack([ w, -z,  y], -1),
        torch.stack([ z,  w, -x], -1),
        torch.stack([-y,  x,  w], -1)], dim=-2)            # (B,4,3)
    return torch.einsum('bij,bi->bj', G, dT_dq)

def rot_grad_matrix(R, dT_dR):
```

```
    """Same quantity from a rotation-matrix gradient.  R, dT_dR: (B,3,3)."""
    M = R.transpose(-1, -2) @ dT_dR
    S = M - M.transpose(-1, -2)
    return torch.stack([S[..., 2, 1], S[..., 0, 2], S[..., 1, 0]], -1)

def aniso_residual(dT_dp, grad_w, St, Sr, eps=1e-12):
    """N(g) = sqrt(St^2 |grad_v|^2 + Sr^2 |grad_w|^2); target is 1.
    |grad_v| = |dT/dp| (world-frame position gradient)."""
    return torch.sqrt(St**2 * (dT_dp**2).sum(-1)
                      + Sr**2 * (grad_w**2).sum(-1) + eps)

# ---- inside the training step ----------------------------------------
# x: (B, 14) = [p_s(3), q_s(4), p_g(3), q_g(4)]      <- ADAPT slices
x = x.requires_grad_(True)
T = model(x)                                          # (B,) or (B,1)
dT_dx = torch.autograd.grad(T.sum(), x, create_graph=True)[0]

ps, qs = x[:, 0:3],  x[:, 3:7]                        # <- ADAPT
pg, qg = x[:, 7:10], x[:, 10:14]                      # <- ADAPT
dps, dqs = dT_dx[:, 0:3],  dT_dx[:, 3:7]
dpg, dqg = dT_dx[:, 7:10], dT_dx[:, 10:14]

N_s = aniso_residual(dps, rot_grad_quat(qs, dqs), St_s, Sr_s)
N_g = aniso_residual(dpg, rot_grad_quat(qg, dqg), St_g, Sr_g)
# St_s, Sr_s, St_g, Sr_g: the existing translational / rotational speeds
# at the start and goal samples, shape (B,).
# Replace the isotropic S*|grad T| in the existing loss by N_s and N_g.
```

For encodings other than quaternions or matrices, use the perturbation form (9):

```
def hat(d):                                           # (B,3) -> (B,3,3)
    z = torch.zeros_like(d[:, 0])
    return torch.stack([torch.stack([z, -d[:, 2], d[:, 1]], -1),
                        torch.stack([d[:, 2], z, -d[:, 0]], -1),
                        torch.stack([-d[:, 1], d[:, 0], z], -1)], -2)

delta = torch.zeros(B, 3, requires_grad=True)
R_pert = R @ (torch.eye(3) + hat(delta))              # exact derivative at 0
T = model(encode(R_pert, p, ...))                     # existing encoder
grad_w = torch.autograd.grad(T.sum(), delta, create_graph=True)[0]
```

5 Path extraction

With the goal fixed and $T(g) = T(g, g_g)$, the time-optimal body velocity is

$$v = -S_t(g)^2\,\nabla_v T, \qquad \omega = -S_r(g)^2\,\nabla_\omega T, \tag{13}$$

and along it $dT/dt = -\big(S_t^2\|\nabla_v T\|^2 + S_r^2\|\nabla_\omega T\|^2\big) = -1$ wherever (3) holds. A Lie–Euler step $g \leftarrow g\exp(\Delta t\,\hat{\xi})$ splits into

$$p \leftarrow p - \Delta t\, S_t(g)^2\,\frac{\partial T}{\partial p}, \qquad R \leftarrow R\,\exp\big(\Delta t\,[\omega]_\times\big), \tag{14}$$

where the position update uses $R\,v = -S_t^2 RR^\top \partial T/\partial p = -S_t^2\,\partial T/\partial p$, so no frame conversion is needed. The rotation update is a *right* (body-frame) multiplication, consistent with how $\nabla_\omega T$ was defined.

```
def quat_mul(a, b):
    w1, x1, y1, z1 = a.unbind(-1); w2, x2, y2, z2 = b.unbind(-1)
    return torch.stack([w1*w2 - x1*x2 - y1*y2 - z1*z2,
                        w1*x2 + x1*w2 + y1*z2 - z1*y2,
                        w1*y2 - x1*z2 + y1*w2 + z1*x2,
                        w1*z2 + x1*y2 - y1*x2 + z1*w2], -1)

def quat_exp(phi):                      # rotation vector -> unit quaternion
    th = phi.norm(dim=-1, keepdim=True)
    # sin(th/2)/th = 0.5*sinc(th/(2*pi)); torch.sinc is safe at 0
    return torch.cat([torch.cos(th / 2),
                      0.5 * torch.sinc(th / (2 * torch.pi)) * phi], -1)

def step(p, q, dT_dp, grad_w, St, Sr, dt):
    p = p - dt * St[:, None]**2 * dT_dp                 # world frame
    q = quat_mul(q, quat_exp(-dt * Sr[:, None]**2 * grad_w))  # body frame
    return p, q / q.norm(dim=-1, keepdim=True)
```

Practical notes:

- Stop when $T(g, g_g)$ falls below a tolerance. Since $dT/dt \approx -1$, the number of steps is roughly $T/\Delta t$.
- A learned T satisfies (3) only approximately. To enforce unit time-rate per step, optionally divide (v, ω) by $N(g)^2$.
- If paths are grown from both endpoints, apply the same step to each end using the gradient with respect to that end's slice.

6 Verification checklist

1. **Rotational gradient vs. finite differences.** For random poses, compare `rot_grad_quat` (or the matrix/perturbation variant) against $\big(T(R\,e^{h[e_j]_\times}, p) - T(R\,e^{-h[e_j]_\times}, p)\big)/(2h)$. They should agree to finite-difference precision.

2. **Analytic solution.** For constant speeds, the block metric is the product metric on $\mathbb{R}^3 \times SO(3)$, and the exact travel time from g_g is

$$T^\star(R, p) = \sqrt{\frac{\|p - p_g\|^2}{S_t^2} + \frac{\theta(R^\top R_g)^2}{S_r^2}}, \tag{15}$$

   where $\theta(\cdot)$ is the rotation angle. Feeding $T^\star$ through the residual code must give $N \equiv 1$, except at $T^\star = 0$ and $\theta = \pi$, where $T^\star$ is not differentiable. This tests the gradient code and the residual together, independently of training.

3. **Isotropic reduction.** Setting $S_t = S_r = S$ must reproduce the original isotropic residual (with the Lie gradient in place of coordinate partials).

4. **Time rate along paths.** Integrating the path ODE on $T^\star$ should decrease $T^\star$ by Δt per step.

Items 1, 2 and 4 were run numerically for these notes (finite-difference ground truth, random poses): the quaternion, matrix and perturbation forms all matched the finite-difference Lie gradient, the analytic residual was 1 ± 10^{-9}, and integrating the path ODE from $T^\star = 1.464$ took 1.444 time units to reach $T^\star = 0.020$, matching the predicted 1.444.

7 Pitfalls

- **Units.** E_1, E_2, E_3 are per unit of position (in whatever units the network sees) and E_4, E_5, E_6 are per radian. S_t must be in position-units per time, S_r in radians per time. If positions are normalized in the code, S_t must be expressed in the normalized units. The ratio S_t/S_r is effectively a length scale that converts radians into distance and sets the rotation–translation trade-off.
- **Speeds near zero.** The residual and the path velocity involve S_t^2 and S_r^2. Keep the same lower clamp the isotropic code uses for S, applied to each block. A block speed of exactly zero makes that direction infinitely costly and the equation degenerate.
- **Frame consistency.** $\nabla_\omega T$ is body-frame; the path update must right-multiply ($R \leftarrow R\exp(\cdot)$). Mixing a body-frame gradient with a world-frame update gives wrong paths for ω while the residual still looks correct (the norms agree).
- **Rodrigues at zero.** Never differentiate through a Rodrigues/exp implementation at exactly zero angle; use the first-order perturbation (9) or the `sinc` form shown in `quat_exp`.
- **Quaternion gradient.** Do not use the raw $\|\partial T/\partial q\|$ as the rotational gradient norm: it includes a radial component and is off by a factor relative to radians. Always map through $G(q)$.

A Derivation sketch

Moving with body twist ξ, the rate of change of T is the plain dot product $\langle \nabla T, \xi \rangle$ with $\nabla T = (\nabla_v T, \nabla_\omega T)$. Writing $\langle \nabla T, \xi \rangle = (W^{-1/2}\nabla T)^\top (W^{1/2}\xi)$ and applying Cauchy–Schwarz,

$$|\langle \nabla T, \xi \rangle| \le \sqrt{\nabla T^\top W^{-1} \nabla T}\, \sqrt{\xi^\top W \xi}, \tag{16}$$

with equality iff $\xi \propto W^{-1}\nabla T$. Since T is a minimum travel time, it cannot change faster than time is spent, $|\langle \nabla T, \xi \rangle| \le \|\xi\|_W$, with equality along optimal paths. Hence the maximum of $|\langle \nabla T, \xi \rangle|/\|\xi\|_W$ equals both 1 and $\sqrt{\nabla T^\top W^{-1} \nabla T}$, which gives

$$\nabla T^\top W(g)^{-1} \nabla T = S_t(g)^2 \, \|\nabla_v T\|^2 + S_r(g)^2 \, \|\nabla_\omega T\|^2 = 1. \tag{17}$$

All steps are pointwise in g, so pose-dependent $S_t(g)$, $S_r(g)$ require no modification. The optimal direction $\xi \propto W^{-1}\nabla T$ gives the path velocity in Section 5.